

Based on the detailed technical requirements document you provided, setting up the backend for this Digital Filing Cabinet (DFC) is a significant but well-defined task. Python is an excellent choice, offering a robust ecosystem of frameworks and libraries to meet these needs.

Here is a comprehensive guide on how to architect and set up this project using a modern Python backend stack.

Core Backend Technology Stack

For a project of this scale, security, and complexity, I recommend a "batteries-included" framework to handle the basics, allowing you to focus on the core business logic.

Web Framework: Django. It's secure, scalable, and comes with a built-in admin, ORM (Object-Relational Mapper), and authentication system, which directly maps to your requirements for RBAC , audit trails , and user management.

API Layer: Django Rest Framework (DRF). This plugs directly into Django and is the industry standard for building powerful, secure, and browsable RESTful APIs that your front end (as specified in the UI section) will consume.

Database: PostgreSQL. It's highly reliable, scales well, and has strong support for relational data, which you'll need for managing the folder hierarchy , user permissions , and audit logs.

Asynchronous Tasks: Celery with RabbitMQ (or Redis) as a message broker. Many actions must not block the user, such as OCR processing , file indexing for search , and applying retention policies. Celery is the standard for handling these background tasks.

Database & File Storage Setup

A critical concept is to separate metadata from the actual files.

1. Database (PostgreSQL)

Your PostgreSQL database will not store the files themselves. It will store all the metadata associated with them, including:

User accounts, roles, and permissions.

The folder hierarchy (e.g., a Folder model with a parent_folder foreign key to itself).

Document metadata (Name, Owner, DocType, Dates, Confidentiality Level, etc.) .

A File model that points to the file's actual location in your file storage.

Tags and their relationships to documents (a many-to-many relationship).

The immutable audit log.

2. File Storage (Object Storage)

Your TRD mentions both "Local Storage" and "Elastic Scalability". The best way to achieve this is with an S3-compatible object storage system.

Recommendation: Use MinIO. It's an open-source, high-performance object store that you can host yourself within your intranet, satisfying the "Local Storage" requirement. It speaks the S3 API, making it "elastically scalable".

Python Integration: Use the `django-storages` library. This allows your Django application to treat MinIO (or AWS S3, etc.) as its default file storage backend. When a user uploads a file, Django will automatically stream it to your MinIO server and save the path to your PostgreSQL database.

Key Functional Components & Python Libraries

Here's how to tackle the main features from the TRD:

1. Security & Access Control (RBAC)

Authentication: Use Django's built-in authentication system for user management. For the API, use a token-based system like JWT (JSON Web Tokens). The Django rest framework-simple jwt library is an excellent choice. This handles your secure login and MFA requirements.

Authorization (RBAC): Implement Role-Based Access Control using Django's Group and Permission models.

Treat Django's Group model as your "Roles" (e.g., "Compliance", "Risk", "HR").

Create custom permissions (e.g., `can_upload_confidential`, `can_delete_folder`) and assign them to these groups.

When a user tries to perform an action, you will check if their assigned group(s) have the required permission for that specific folder or document.

2. Search & Retrieval

Your requirement for "Full-Text Search" and "Advanced Filtering" goes beyond what PostgreSQL's basic search can do, especially with the volume of documents implied.

Recommendation: Elasticsearch (or its open-source fork, OpenSearch).

Python Integration: Use the `django-elasticsearch-dsl` library.

How it works:

When a file is uploaded, a Celery background task is triggered.

This task extracts text from the document (using libraries like `pypdf2` for PDFs or `python-docx` for Word docs).

For scanned images/PDFs, the task first runs them through Tesseract (using the `pytesseract` library) to satisfy the OCR requirement.

The extracted text and all its metadata (tags, owner, date, etc.) are sent to Elasticsearch to be indexed.

Your API's search endpoint will then query Elasticsearch, not your database, for lightning-fast, relevant results.

3. Audit Trails

Create an AuditLog model in Django.

This model should be append-only. It will store the user, the action performed (e.g., "CREATE", "DELETE", "VIEW"), the timestamp, and the target object (e.g., folder ID or document ID).

You can use Django's Signals or override the `.save()` and `.delete()` methods on your models to automatically create an audit log entry whenever a relevant action occurs.

Suggested Backend Project Structure

Here is a logical way to structure your Django project to keep it maintainable:

dfc_backend/

```
├── manage.py      # Django's management script
├── config/        # Project-level settings (settings.py, urls.py, wsgi.py)
├── apps/          # Root folder for your applications
│   ├── users/     # Manages user accounts, profiles, auth, and RBAC
│   │   ├── models.py  (CustomUser model, Profile)
│   │   ├── serializers.py
│   │   └── views.py   (Login, Logout, User/Role management API)
│   │
│   ├── documents/  # Core app for files, folders, and metadata
│   │   ├── models.py  (Folder, Document, Tag, Metadata)
│   │   ├── serializers.py
│   │   ├── views.py   (File/Folder CRUD operations, upload API)
│   │   └── signals.py (Triggers audit logs, search indexing)
```

```

| |
| |─ search/      # Handles search indexing and queries
| |   └─ documents.py (Defines Elasticsearch index structure)
| |       └─ views.py  (The search API endpoint)
| |
| |
| |─ audit/        # Manages the immutable audit trail
| |   └─ models.py  (AuditLog model)
| |       └─ serializers.py
| |
| |
| |─ workflows/    # Handles automation and retention policies
| |   └─ tasks.py   (Celery tasks for OCR, indexing, retention)
| |
| |
└─ requirements.txt # List of Python libraries (Django, django rest framework, etc.)

```

This structure separates concerns, making the complex system easier to build, test, and maintain, aligning perfectly with the developer roadmap suggested in your document